SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
॥वसुधैव कुटुम्बकम्॥ Re-Accredited by NAAC with 'A++' Grade

Subject: Programming with Java

ASSIGNMENT No : 6

TITLE : Write a Java program to demonstrate the use of an inner class, an anonymous class.

Name: Arpan Gupta	Class: CSE A2
PRN: 25070122046	Batch: 2025-29

Problem Statement
Write a Java program to demonstrate the use of an inner class, an anonymous class.

Learning Objectives
To implement inner classes and anonymous classes for improving encapsulation and event handling in Java.

Theory
An inner class is a class defined inside another class and has access to all members of the enclosing class, including private members. Inner classes help improve code organization and logical grouping of related classes. Anonymous classes are unnamed inner classes that are instantiated only once and are mainly used for implementing interfaces or abstract classes. They simplify code by eliminating the need for separate class definitions. These concepts are widely used in Java GUI programming and event handling.

Outcome

InnerClassExample.java

Something > src > lab > InnerClassExample.java > InnerClassExample

```java
package lab;

/**
 * This program demonstrates all 4 types of inner classes in Java:
 * 1. Member Inner Class (non-static nested class)
 * 2. Static Nested Class
 * 3. Local Inner Class (defined inside a method)
 * 4. Anonymous Inner Class (defined and instantiated in a single expression)
 */

public class InnerClassExample {

    // ============================================================
    // 1. MEMBER INNER CLASS (non-static nested class)
    // ============================================================
    // A regular inner class that has access to all members of the outer class,
    // including private members. It must be instantiated using an instance of
    // the outer class.
    private String outerMessage = "Hello from Outer Class!";

    class MemberInner {
        void display() {
            // Can access private members of the outer class
            System.out.println("  [Member Inner Class] Accessing outer class private field: \"" + outerMessage + "\"");
            System.out.println(x: "  [Member Inner Class] This inner class is tied to an instance of OuterClass.");
        }
    }

    // ============================================================
    // 2. STATIC NESTED CLASS
    // ============================================================
    // A static nested class behaves like a top-level class but is nested for
    // packaging convenience. It CANNOT access non-static members of the outer
    // class directly. It does NOT require an instance of the outer class.
    private static String staticOuterMessage = "Hello from Static Outer Context!";

    static class StaticNested {
        void display() {
            // Can only access static members of the outer class
            System.out.println("  [Static Nested Class] Accessing static outer field: \"" + staticOuterMessage + "\"");
            System.out.println(x: "  [Static Nested Class] This class does NOT need an instance of OuterClass.");
        }
    }

```

```java
    // ============================================================
    // Method to demonstrate LOCAL INNER CLASS
    // ============================================================
    public void demonstrateLocalInnerClass() {
        // Local variable (must be effectively final or explicitly final in Java 8+)
        final String localMessage = "Hello from Local Context!";

        // 3. LOCAL INNER CLASS
        // A class defined inside a method. It is only accessible within that method.
        // It can access the enclosing method's final/effectively-final local variables.
        class LocalInner {
            void display() {
                System.out.println("  [Local Inner Class] Accessing local variable: \"" + localMessage + "\"");
                System.out.println(x: "  [Local Inner Class] This class is defined inside a method and is only");
                System.out.println(x: "                      visible within the scope of this method.");
            }
        }

        // Instantiate and use the local inner class
        LocalInner local = new LocalInner();
        local.display();
    }
```

```java
    // ============================================================
    // Method to demonstrate ANONYMOUS INNER CLASS
    // ============================================================
    public void demonstrateAnonymousInnerClass() {
        // 4. ANONYMOUS INNER CLASS
        // A class without a name that is defined and instantiated in a single expression.
        // It is typically used to override methods of a class or interface on the fly.

        // Create an anonymous inner class that extends the Thread class
        Thread anonymousThread = new Thread() {
            @Override
            public void run() {
                System.out.println(x: "  [Anonymous Inner Class] Running inside an anonymous Thread subclass!");
                System.out.println(x: "  [Anonymous Inner Class] This class is defined and instantiated in one go.");
            }
        };

        // Alternatively, using a Runnable interface
        Runnable anonymousRunnable = new Runnable() {
            @Override
            public void run() {
                System.out.println(x: "  [Anonymous Inner Class] Running inside an anonymous Runnable implementation!");
            }
        };

        anonymousThread.start();
        new Thread(anonymousRunnable).start();

        // Give threads time to print output
        try {
            Thread.sleep(millis: 100);
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
```

```
104     // ============================================================
105     // MAIN METHOD
106     // ============================================================
        Run | Debug
107     public static void main(String[] args) {
108         System.out.println(x: "============================================");
109         System.out.println(x: "   INNER CLASS DEMONSTRATION IN JAVA");
110         System.out.println(x: "============================================\n");
111
112         // --- 1. Member Inner Class ---
113         System.out.println(x: "1. MEMBER INNER CLASS:");
114         System.out.println(x: "   (A non-static class defined inside another class)");
115         // To instantiate a member inner class, you need an instance of the outer class
116         InnerClassExample outer = new InnerClassExample();
117         InnerClassExample.MemberInner memberInner = outer.new MemberInner();
118         memberInner.display();
119         System.out.println();
120
121         // --- 2. Static Nested Class ---
122         System.out.println(x: "2. STATIC NESTED CLASS:");
123         System.out.println(x: "   (A static class defined inside another class)");
124         // Static nested class can be instantiated without an outer class instance
125         InnerClassExample.StaticNested staticNested = new InnerClassExample.StaticNested();
126         staticNested.display();
127         System.out.println();
128
129         // --- 3. Local Inner Class ---
130         System.out.println(x: "3. LOCAL INNER CLASS:");
131         System.out.println(x: "   (A class defined inside a method)");
132         outer.demonstrateLocalInnerClass();
133         System.out.println();
134
135         // --- 4. Anonymous Inner Class ---
136         System.out.println(x: "4. ANONYMOUS INNER CLASS:");
137         System.out.println(x: "   (A class without a name, defined and instantiated together)");
138         outer.demonstrateAnonymousInnerClass();
139         System.out.println();
140
141         System.out.println(x: "============================================");
142         System.out.println(x: "   END OF DEMONSTRATION");
143         System.out.println(x: "============================================");
144     }
145 }
```

Conclusion

This assignment has demonstrated the concept of inner classes and anonymous classes, as well as demonstrated applications wherein these concepts are utilized.

Exercise

1.Create a **Vehicle program** where an inner class displays vehicle details and anonymous class performs an action.

Vehicle.java

Something > src > lab > Vehicle.java > { } lab

```java
package lab;

/**
 * Vehicle program demonstrating:
 * 1. Member Inner Class - VehicleDetails displays vehicle details
 * 2. Anonymous Inner Class - performs an action
 */
public class Vehicle {

    // ---------- Outer class fields ----------
    private String brand;
    private String model;
    private int year;
    private int speed;
    private String engineType;

    // ---------- Constructor ----------
    public Vehicle(String brand, String model, int year, String engineType) {
        this.brand = brand;
        this.model = model;
        this.year = year;
        this.speed = 0;
        this.engineType = engineType;
    }

    // ---------- Getter methods ----------
    public String getBrand() {
        return brand;
    }

    public String getModel() {
        return model;
    }

    public int getYear() {
        return year;
    }

    public int getSpeed() {
        return speed;
    }

    public String getEngineType() {
        return engineType;
    }
```

```
    // ============================================================
    // 1. MEMBER INNER CLASS - VehicleDetails
    // ============================================================
    /**
     * Inner class that displays all vehicle details.
     * It has access to private fields of the outer Vehicle class.
     */
    class VehicleDetails {
        void display() {
            System.out.println("===== VEHICLE DETAILS =====");
            System.out.println("Brand      : " + brand);
            System.out.println("Model      : " + model);
            System.out.println("Year       : " + year);
            System.out.println("Engine     : " + engineType);
            System.out.println("Speed      : " + speed + " km/h");
            System.out.println("============================");
        }
    }
```

```
// ============================================================
// 2. ANONYMOUS INNER CLASS - performs an action
// ============================================================
/**
 * Interface for vehicle actions.
 */
interface VehicleAction {
    void performAction();
}

/**
 * Demonstrates the use of an anonymous inner class.
 * An anonymous class implements the VehicleAction interface
 * and overrides performAction() on the fly.
 */
public void demonstrateAction() {
    // Anonymous inner class implementing VehicleAction
    VehicleAction action = new VehicleAction() {
        @Override
        public void performAction() {
            System.out.println(">> [Anonymous Inner Class] Action: Vehicle is accelerating!");
            System.out.println(">> [Anonymous Inner Class] Current speed: " + speed + " km/h");
            System.out.println(">> [Anonymous Inner Class] Engine (" + engineType + ") roars to life!");
            System.out.println(">> [Anonymous Inner Class] Vroom! Vroom!");
        }
    };

    action.performAction();
}
```

```java
    // ============================================================
    // MAIN METHOD
    // ============================================================
    Run | Debug
    public static void main(String[] args) {
        System.out.println(x: "========================================");
        System.out.println(x: "   VEHICLE PROGRAM - INNER CLASS DEMO");
        System.out.println(x: "========================================\n");

        // Create a Vehicle instance
        Vehicle myVehicle = new Vehicle(brand: "Toyota", model: "Camry", year: 2024, engineType: "V6 Petrol");

        // --- 1. Member Inner Class: Display Vehicle Details ---
        System.out.println(x: "1. MEMBER INNER CLASS (VehicleDetails):");
        System.out.println(x: "   Displays all vehicle information\n");

        // Instantiate the inner class using outer class instance
        Vehicle.VehicleDetails details = myVehicle.new VehicleDetails();
        details.display();

        System.out.println();

        // --- 2. Anonymous Inner Class: Perform Action ---
        System.out.println(x: "2. ANONYMOUS INNER CLASS:");
        System.out.println(x: "   Performs an action by implementing VehicleAction interface\n");

        myVehicle.demonstrateAction();

        System.out.println();
        System.out.println(x: "========================================");
        System.out.println(x: "   END OF VEHICLE PROGRAM");
        System.out.println(x: "========================================");
    }
}
```

2. Develop a **Food Delivery Application** where an inner class handles order details and anonymous classes handle delivery status updates.

FoodDelivery.java

Something > src > lab > FoodDelivery.java > FoodDelivery

```java
package lab;

import java.util.ArrayList;

/**
 * Food Delivery Application demonstrating:
 * 1. Member Inner Class - OrderDetails displays complete order information
 * 2. Anonymous Inner Classes - Handle delivery status updates at various stages
 */
public class FoodDelivery {

    // ---------- Outer class fields ----------
    private String restaurantName;
    private String customerName;
    private String orderId;
    private String deliveryAddress;
    private String orderStatus;
    private ArrayList<String> items;
    private double totalAmount;
    private String estimatedDeliveryTime;

    // ---------- Constructor ----------
    public FoodDelivery(String restaurantName, String customerName, String orderId,
                        String deliveryAddress, String estimatedDeliveryTime) {
        this.restaurantName = restaurantName;
        this.customerName = customerName;
        this.orderId = orderId;
        this.deliveryAddress = deliveryAddress;
        this.estimatedDeliveryTime = estimatedDeliveryTime;
        this.orderStatus = "Order Pending";
        this.items = new ArrayList<>();
        this.totalAmount = 0.0;
    }

    // ---------- Methods ----------
    /**
     * Adds an item to the order.
     * @param item Name of the menu item
     * @param price Price of the item
     */
    public void addItem(String item, double price) {
        items.add(item);
        totalAmount += price;
        System.out.println("   [Added] " + item + " - $" + String.format(format: "%.2f", price));
    }
```

```java
    /**
     * Updates the current order status.
     * @param status New status string
     */
    public void updateOrderStatus(String status) {
        this.orderStatus = status;
    }

    // ============================================================
    // 1. MEMBER INNER CLASS - OrderDetails
    // ============================================================
    /**
     * Inner class that displays complete order information.
     * It has access to all private fields of the outer FoodDelivery class.
     */
    class OrderDetails {
        void displayOrderSummary() {
            System.out.println(x: "======================================");
            System.out.println(x: "         ORDER SUMMARY");
            System.out.println(x: "======================================");
            System.out.println("Order ID      : " + orderId);
            System.out.println("Restaurant    : " + restaurantName);
            System.out.println("Customer      : " + customerName);
            System.out.println(x: "Items Ordered :");
            if (items.isEmpty()) {
                System.out.println(x: "   (No items added yet)");
            } else {
                for (int i = 0; i < items.size(); i++) {
                    System.out.println("   " + (i + 1) + ". " + items.get(i));
                }
            }
            System.out.println("Total Amount  : $" + String.format(format: "%.2f", totalAmount));
            System.out.println(x: "======================================");
        }

        void displayDeliveryInfo() {
            System.out.println(x: "======================================");
            System.out.println(x: "       DELIVERY INFORMATION");
            System.out.println(x: "======================================");
            System.out.println("Delivery Address : " + deliveryAddress);
            System.out.println("Estimated Time   : " + estimatedDeliveryTime);
            System.out.println("Current Status   : " + orderStatus);
            System.out.println(x: "======================================");
        }
```

```
93      // ============================================================
94      // 2. INTERFACE - DeliveryStatusCallback
95      // ============================================================
96      /**
97       * Callback interface for delivery status updates.
98       * Implemented by anonymous inner classes to handle different
99       * stages of the delivery lifecycle.
100      */
101     interface DeliveryStatusCallback {
102         void onStatusUpdate(String newStatus, String message);
103     }
104
105     // ============================================================
106     // 3. Method demonstrating ANONYMOUS INNER CLASSES
107     // ============================================================
108     /**
109      * Simulates the complete delivery lifecycle using anonymous inner classes.
110      * Each stage of delivery is handled by a separate anonymous class
111      * implementing the DeliveryStatusCallback interface.
112      */
113     public void simulateDeliveryLifecycle() {
114         System.out.println(x: "\n========== DELIVERY LIFECYCLE ==========\n");
115
116         // ------------- Stage 1: Order Confirmed -------------
117         System.out.println(x: ">>> Stage 1: Order Confirmation");
118         DeliveryStatusCallback confirmedCallback = new DeliveryStatusCallback() {
119             @Override
120             public void onStatusUpdate(String newStatus, String message) {
121                 updateOrderStatus(newStatus);
122                 System.out.println("   [Anonymous] Status: " + orderStatus);
123                 System.out.println("   [Anonymous] Message: " + message);
124                 System.out.println(x: "   [Anonymous] Your order has been received and is being processed.\n");
125             }
126         };
127         confirmedCallback.onStatusUpdate(newStatus: "Order Confirmed",
128             message: "Thank you for your order! Your food will be prepared shortly.");
129
```

```
130         // ------------ Stage 2: Preparing Food ------------
131         System.out.println(x: ">>> Stage 2: Food Preparation");
132         DeliveryStatusCallback preparingCallback = new DeliveryStatusCallback() {
133             @Override
134             public void onStatusUpdate(String newStatus, String message) {
135                 updateOrderStatus(newStatus);
136                 System.out.println("    [Anonymous] Status: " + orderStatus);
137                 System.out.println("    [Anonymous] Message: " + message);
138                 System.out.println(x: "    [Anonymous] The chef is preparing your meal with fresh ingredients.\n");
139             }
140         };
141         preparingCallback.onStatusUpdate(newStatus: "Preparing",
142             message: "Your order is being prepared by our top chefs!");
143 
144         // ------------ Stage 3: Out for Delivery ------------
145         System.out.println(x: ">>> Stage 3: Out for Delivery");
146         DeliveryStatusCallback outForDeliveryCallback = new DeliveryStatusCallback() {
147             @Override
148             public void onStatusUpdate(String newStatus, String message) {
149                 updateOrderStatus(newStatus);
150                 System.out.println("    [Anonymous] Status: " + orderStatus);
151                 System.out.println("    [Anonymous] Message: " + message);
152                 System.out.println("    [Anonymous] Your delivery partner is on the way to " + deliveryAddress + "\n");
153             }
154         };
155         outForDeliveryCallback.onStatusUpdate(newStatus: "Out for Delivery",
156             message: "Your delivery partner has picked up the order and is heading your way!");
157 
```

```java
        // ------------- Stage 4: Delivered -------------
        System.out.println(x: ">>> Stage 4: Delivered");
        DeliveryStatusCallback deliveredCallback = new DeliveryStatusCallback() {
            @Override
            public void onStatusUpdate(String newStatus, String message) {
                updateOrderStatus(newStatus);
                System.out.println("   [Anonymous] Status: " + orderStatus);
                System.out.println("   [Anonymous] Message: " + message);
                System.out.println(x: "   [Anonymous] Bon appétit! 🎉\n");
            }
        };
        deliveredCallback.onStatusUpdate(newStatus: "Delivered",
            message: "Your order has been delivered! Enjoy your meal!");

        System.out.println(x: "==========================================\n");
    }

    // ============================================================
    // MAIN METHOD
    // ============================================================
    Run | Debug
    public static void main(String[] args) {
        System.out.println(x: "==========================================");
        System.out.println(x: "   FOOD DELIVERY APPLICATION");
        System.out.println(x: "   Inner Class & Anonymous Class Demo");
        System.out.println(x: "==========================================\n");

        // Create a FoodDelivery instance
        FoodDelivery order = new FoodDelivery(
            restaurantName: "Pizza Paradise",
            customerName: "Arpan Gupta",
            orderId: "ORD-2024-001",
            deliveryAddress: "123, Main Street, New Delhi",
            estimatedDeliveryTime: "30-40 minutes"
        );

        // Add items to the order
        System.out.println(x: "--- Adding Items to Order ---");
        order.addItem(item: "Margherita Pizza", price: 12.99);
        order.addItem(item: "Garlic Breadsticks", price: 4.99);
        order.addItem(item: "Coke (500ml)", price: 2.50);
        System.out.println();
```

```
199
200         // --- 1. Member Inner Class: Display Order Details ---
201         System.out.println(x: "1. MEMBER INNER CLASS (OrderDetails):");
202         System.out.println(x: "   Displays complete order and delivery information\n");
203
204         // Instantiate the inner class using outer class instance
205         FoodDelivery.OrderDetails details = order.new OrderDetails();
206         details.displayOrderSummary();
207         System.out.println();
208         details.displayDeliveryInfo();
209
210         System.out.println();
211
212         // --- 2. Anonymous Inner Classes: Delivery Lifecycle ---
213         System.out.println(x: "2. ANONYMOUS INNER CLASSES:");
214         System.out.println(x: "   Each delivery stage uses a separate anonymous class\n");
215
216         order.simulateDeliveryLifecycle();
217
218         // --- Final status check ---
219         System.out.println(x: "--- Final Order Status ---");
220         details.displayDeliveryInfo();
221
222         System.out.println(x: "\n==========================================");
223         System.out.println(x: "   END OF FOOD DELIVERY DEMO");
224         System.out.println(x: "==========================================");
225     }
226 }
```